

Home Lab Improvement Plan

One plan from two reviews: what to fix, in what order, and how to prove it

This plan merges the deep technical review with my earlier improvement plan, both written against commit 08c01c7. The deep review is the base, because all eleven of its reproductions were re-run and confirmed. The earlier plan adds the macOS operations work and effort estimates. The architecture stays small: SQLite, one Mac, one heavy model slot and a trusted control plane. The work is making its guarantees hold under failure and hostile input, then shipping one useful workflow.

ITEMS	RELEASE BLOCKERS	CORE EFFORT	REPRODUCED	NEEDS THE M6
46 38 core, 8 optional	10 all in phase 1	~68 days about 14 weeks full-time	11 of 11 re-run and confirmed	Phase 5 everything else runs now

Start here

Until phase 1 passes, run the lab only with dummy data, isolated workspaces and drafts that a person reviews. Phases run in order, with hygiene alongside the first. Each phase ends at a gate, and authority grows only after a gate passes.

Your first week

Day 1, about one day of quick wins. 1.9 durable commits and the SQLite version check, 2.1 lockfile, 2.2 CI pinning, 2.5 GitHub security and 2.6 correcting the claims in the docs.

Days 2 to 5, the foundations. 1.6 one transaction per state change, 1.3 lease tokens and 1.1 policy on every tool call, then begin 2.4 by turning R01 to R11 into failing regression tests.

Weeks 2 to 4. 1.2 identity, 1.4 action intents, 1.5 race-safe files, 1.7 retries, 1.8 stopping work and 1.10 request limits.

How to work through it

- One item per pull request, with the item number in the title.
- Add the failing regression first, change the enforcement path, then prove the real effect is blocked.
- Update ARCHITECTURE, SECURITY and module comments in the same pull request (item 2.6).
- Tick the box on the card and in the checklist at the back.

Roadmap and gates

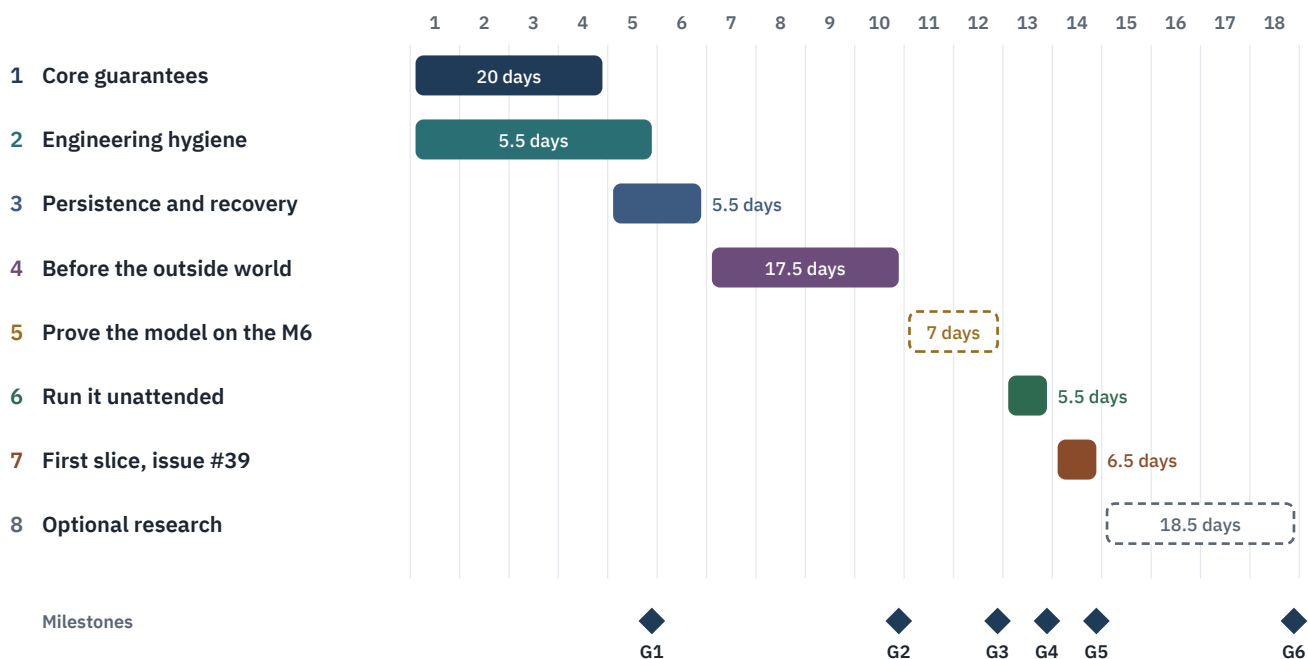


Figure 1. Roadmap at full-time pace, in weeks. Bars show focused days. Dashed bars need the Mac mini (phase 5) or are optional (phase 8). Move phase 5 earlier if the machine arrives sooner.

GATE	WHEN	MUST PASS	WHAT IT ALLOWS
G1 Core guarantees hold	End of week 5	Phase 1 done, R01 to R11 regressions passing	Development on real tasks with dummy data
G2 Safe to read outside	End of week 10	Phases 3 and 4 done, attack tests passing	Starting issue #39
G3 Model measured	End of week 12	Phase 5 done, ADR 0001 updated	Committing to the model
G4 Runs unattended	End of week 13	Phase 6 done, restore drill passed	Leaving it running, and P3's window
G5 First slice works	End of week 14	Phase 7 done, one event gives one reviewed draft	A measured pilot on real tasks
G6 Research-ready	Week 18 or later	Chosen phase 8 items, each with a measured gain	Publishing and optional features

What changed from version 1

CHANGE	DETAIL
Added from the deep review	Identity binding, lease tokens with a singleton supervisor, immutable action intents, race-safe file handling, side-effect-aware retries, the SQLite version gate, a narrow fetch gateway, content-addressed artifacts, the issue #39 slice and the optional research items.
Corrected in my plan	The audit chain now needs an outside checkpoint. The credential proxy became one connector per destination. Apple container is limited to Linux-compatible code.
Kept from my plan	fullfsync on macOS, FileVault and power decisions, external alerts, the macOS update policy, the git-ignored lockfile, CI hardening, pre-registration, DOIs and effort estimates.

Phase 1 Make the core guarantees true

Weeks 1 to 4 · about 20 focused days · 10 items · Priority P0, release blockers

Each item closes a gap reproduced in the current code. Until all ten pass their tests, run the lab only with dummy data, isolated workspaces and drafts that a person reviews.

☐ 1.1 Enforce policy on every tool call

Effort: 1.5 days · Depends on: None · From: HL01, R01, v1 1.4 · [Start here](#)

Why. TOOL_TIERS is only a label. submit() checks the tool allowlist, then runs the tool without asking policy. A test deleted a workspace file with no approval, and a policy spy was never called.

Steps

1. Make the broker the only dispatch point, driven by a trusted tool registry.
2. For every call, check the tool, resource scope, current task and attempt, input origin, budget and any exact-action approval.
3. Refuse on an unknown tool, an unavailable policy engine or a failed audit write.
4. When an approval is missing, park the task and open a request that shows the exact action.

Done when. A task that holds a tool name still cannot run an approval-required operation without a matching grant, and the test checks that the file or remote effect did not happen.

Where: lab/broker.py:67 (TOOL_TIERS), 172 (submit)

☐ 1.2 Bind identity to a trusted execution context

Effort: 3 days · Depends on: 1.1 · From: HL02, R02

Why. ToolRequest.worker and task_id are plain text that nobody checks. Changing the worker label gave access to another task's workspace. Handlers are arbitrary Python running inside the supervisor.

Steps

1. Create the execution context in trusted code: task, agent, attempt and lease generation.
2. Pass it through a supervised worker process or an OS-authenticated local channel, never inside the request.
3. Run workers under an account and mounts that cannot reach the database, policy files or credentials.
4. Allow only reviewed handler code, never model-generated Python.

Done when. Forged task ids and worker labels fail. A test worker cannot read the queue database, change policy, open another workspace or create a more privileged broker.

□ 1.2 Bind identity to a trusted execution context

Trade-off. A separate worker process adds plumbing. It is the only way hostile handler code cannot reach the supervisor's memory.

Where: lab/broker.py:79 (ToolRequest), lab/supervisor.py (register)

□ 1.3 Replace owner-name fencing with lease tokens

Effort: 2 days · Depends on: None · From: HL03, R03, R04, v1 1.3 · [Start here](#)

Why. The fencing check skips any owner that never held a lease, so a second client marked another owner's running task as done. A stale process sharing an owner name also completed work after recovery and released the replacement's lease.

Steps

1. Issue a lease token at claim time: task id, lease id, a generation that rises on every claim, and a process instance id.
2. Require the exact token for start, renew, succeed, fail, park, release and every broker call.
3. Check the token and change the state in one transaction, and check the affected row count.
4. Remove the never-owned exemption. Keep admin cancel and recovery as separate APIs.
5. Take a host-wide singleton lock before recovery, so a second supervisor refuses to start.

```
@dataclass(frozen=True)
class LeaseToken:
    task_id: str
    lease_id: str
    generation: int      # rises on every claim
    instance_id: str     # unique per process start
```

Done when. A never-owner write, a stale generation, a reused owner name, an expired renewal and a late result from an old worker all raise LeaseLost and leave the state unchanged. The loser cannot release the replacement lease.

Where: lab/queue.py (_require_lease, owns_lease, recover)

□ 1.4 Approve an immutable action intent

Effort: 2.5 days · Depends on: 1.1, 1.3 · From: HL04, R08

Why. The approval hash covers the task payload, not file contents or later tool calls. Consumption only checks that a grant is unused, so a grant that expires between lookup and use is still spent.

Steps

1. Build the intent in trusted code before review: task, lease generation, tool, policy version, canonical target and arguments, content hashes, preconditions, limits and expiry.
2. Show the reviewer that exact object and hash a defined serialization of it.
3. Reserve the intent in one update that checks every condition, and record the attempt in the same transaction.
4. If content or destination state changes, require a new intent.

```
UPDATE approvals SET consumed_at = :now
WHERE id = :id AND state = 'granted' AND consumed_at IS NULL
AND expires_at > :now AND action_hash = :hash
```

Done when. Changing one argument, recipient, content hash, policy version or precondition invalidates the grant. Expiry at the moment of use is refused. Two consumers cannot reserve the same intent.

Where: lab/policy.py (action_hash, _consume)

□ 1.5 Make file handling race-safe and protect the profile

Effort: 2.5 days · Depends on: 1.2 design · From: HL05, R09, R10, v1 1.6 · [Start here](#)

Why. A directory swapped for a symlink after the path check made a write land outside the workspace. A symlink planted at `.sandbox.sb` made the supervisor overwrite a file outside the workspace before any command started.

Steps

1. Write the profile in a controller-owned directory with exclusive creation and private permissions, or pass it inline with `sandbox-exec -p`.
2. Use descriptor-relative file operations that refuse symlinks in every path component, or run the file helper itself inside the sandbox. A second `resolve()` is not a race fix.
3. Generate workspace names internally, use private modes and one owner per attempt, and refuse repeat opens and deletion of the workspace root.
4. Escape or reject characters that change the meaning of the profile text.
5. Always deny writes to `.git/hooks` and shell start-up files inside the workspace.

Done when. On the Mac, symlink and directory-swap races against reads, writes, deletes and artifact collection leave outside canary files unchanged, and a pre-planted `.sandbox.sb` link changes nothing.

Where: `lab/broker.py` (`Workspace.resolve()`), `lab/sandbox.py:180`

□ 1.6 Commit each state change with its audit record

Effort: 2 days · Depends on: 1.3 · From: HL06, R05, v1 1.1 · [Start here](#)

Why. Each step of a transition saves on its own. An injected failure while writing the succeeded event left the task succeeded, with no event and a live lease. A crash mid-park left a lease that blocked the task for good.

Steps

1. Run create, claim, start, complete, fail, request, grant, deny and recover each as one transaction.
2. Make every transition a conditional update on the expected prior state, and check the row count.
3. Roll back explicitly when the event or lease update fails.
4. Keep model calls and network requests outside write transactions.
5. Keep all SQL in the queue and policy service, not the CLI, and retry busy errors a bounded number of times.

```
@contextmanager
def tx(conn):
    conn.execute("BEGIN IMMEDIATE")
    try:
        yield
    except BaseException:
        conn.execute("ROLLBACK")
        raise
    conn.execute("COMMIT")
```

Done when. Fault injection before and after each statement leaves state, active lease and event in agreement after restart. Competing start, complete and recovery calls have exactly one winner.

Where: `lab/queue.py:182, 320 to 360`, `lab/policy.py` (`_release_task`)

□ 1.7 Retry only when the outcome is known

Effort: 2.5 days · Depends on: 1.4, 1.6 · From: HL07, R06

Why. `fail()` requeues a task while attempts remain, even one marked unsafe to repeat. A timeout after an email went out could send it again. Leasing also spends attempts before policy runs, so waiting for an approval uses up the retry budget.

Steps

1. Classify outcomes: read-only, no side effect started, remote result unknown, non-idempotent change, and invalid input.
2. Give every external operation a stable id reused across retries, with a fingerprint of its parameters.
3. Keep an operation journal with the states prepared, executing, confirmed and uncertain.
4. Hold non-idempotent and uncertain operations for reconciliation or review instead of retrying.
5. Count execution attempts separately from lease claims and approval waits.

Done when. An operation that succeeds remotely while its response is lost is reconciled or held after restart, never sent twice. Permanent errors and exhausted budgets stop cleanly.

Where: `lab/queue.py:325 (fail), lease() attempts`

□ 1.8 Stop the work, not only its result

Effort: 2 days · Depends on: 1.2, 1.3 · From: HL08, R07, v1 1.2, v1 1.3

Why. A policy error killed a worker, yet `run()` returned normally with the task still leased. When lease renewal fails, the task keeps running. `stop()` only stops new leasing. A blocking tool call can also freeze the event loop and starve the lease heartbeat.

Steps

1. Keep a handle to every worker and child process.
2. On lease loss, cancellation or emergency stop, revoke broker authority first, then drain briefly and end the whole process group.
3. Restart failed workers under a bounded policy, record why, and report the service unhealthy when it cannot recover.
4. Run blocking tool calls off the event loop, through `asyncio` subprocesses or a worker process.

Done when. After an injected policy error the worker recovers and the task is released. After lease loss or a stop, no later tool call succeeds and no child process survives.

Where: `lab/supervisor.py:126, 135, 164 to 177`

□ 1.9 Durable commits on a safe SQLite

Effort: 0.5 day · Depends on: None · From: HL14, v1 1.5 · [Start here](#)

Why. WAL with `synchronous=NORMAL` can lose recent commits in a power cut, and plain `fsync` on macOS is not a full flush. SQLite 3.7.0 to 3.51.2 also carry a rare corruption bug, fixed in 3.51.3 with backports 3.44.6 and 3.50.7, and some uv-managed Python builds shipped an affected version.

Steps

1. Set `synchronous=FULL`, `fullfsync=ON`, `checkpoint_fullfsync=ON`, foreign keys and a busy timeout on every connection.
2. At start-up, read `sqlite3.sqlite_version` and refuse to run on an affected release.
3. Record the version in every run's provenance, and keep the database on the internal disk.

❑ 1.9 Durable commits on a safe SQLite

```
FIXED = ((3, 51, 3), (3, 50, 7), (3, 44, 6))
v = tuple(int(x) for x in sqlite3.sqlite_version.split("."))
safe = v >= (3, 51, 3) or any(v[:2] == f[:2] and v >= f for f in FIXED)
conn.execute("PRAGMA synchronous = FULL")
conn.execute("PRAGMA fullfsync = ON")
```

Done when. Every connection reports FULL with fullfsync on, and the supervisor refuses to start on an affected SQLite.

Trade-off. Each commit waits for the disk, which costs milliseconds per task.

Where: lab/schema.sql:21, lab/queue.py:120

❑ 1.10 Clean environment and strict limits for every request

Effort: 1.5 days · Depends on: 1.1, 1.8 · From: HL12, R11, v1 1.6

Why. Sandboxed commands inherit the supervisor's full environment, so any token in it reaches them. The model can request any timeout, and captured output sits in memory without a limit.

Steps

1. Launch with an explicit minimal environment: PATH, HOME set to the workspace, and LANG.
2. Validate parameters strictly: known fields only, a finite positive timeout, string arguments only.
3. Clamp timeouts, stream output with a size cap, and cap reads, listings and workspace bytes.
4. Limit tool calls, child tasks, elapsed time and inference tokens per task.

Done when. No supervisor variable reaches a command, an output flood or fork bomb cannot freeze the controller, and malformed arguments are refused.

Where: lab/sandbox.py:188, lab/broker.py:242

Phase 2 Engineering hygiene

Weeks 1 to 5, alongside phase 1 · about 5.5 focused days · 6 items · Priority P1

Cheap changes that make every later change safer. Most take an hour or two, so fit them around phase 1.

❑ 2.1 Commit the lockfile and pinned Python

Effort: 1 hour · Depends on: None · From: HL17, v1 2.1 · **Start here**

Why. .gitignore lists uv.lock and .python-version, so the mini cannot be rebuilt to the same state. A Python pin alone also does not prove the SQLite inside it is safe (see 1.9).

Steps

1. Remove both lines from .gitignore.
2. On the mini, pin the exact patch version with uv python pin, then run uv lock and commit both.
3. Install with uv sync --locked in CI and deployment.
4. Record uv, SQLite, MLX, model and sandbox-template versions for every run.

```
uv python pin <exact 3.13.x version on the mini>
uv lock
uv sync --locked
```

Done when. CI installs only from the lockfile and fails when it is out of date.

□ 2.1 Commit the lockfile and pinned Python

Where: `.gitignore:17 to 18`, `docs/decisions/0002-python-runtime.md`

□ 2.2 Harden the CI pipeline

Effort: 2 hours · Depends on: 2.1 · From: HL17, v1 2.2 · [Start here](#)

Why. Actions are pinned by movable tags, and macos-latest moved to macos-26 in June 2026, so the test platform can change without notice.

Steps

1. Pin every action to a full commit SHA with a version comment, using `pinact`.
2. Scan the workflows with `zizmor`, and replace `macos-latest` with `macos-26`.
3. Check the wheel installs in a clean environment.
4. Keep pull-request jobs away from credentials and from any Mac runner.

```
pinact run
zizmor .github/workflows
```

Done when. `zizmor` reports no unpinned actions, and the workflow names its macOS version.

Where: `.github/workflows/check.yml`

□ 2.3 Type checks, coverage floors and a safety marker

Effort: 1.5 days · Depends on: 2.1 · From: v1 2.3, HL30

Why. `mypy` in strict mode reports 18 issues. The CI safety check counts test names, and counting names does not prove a property is enforced.

Steps

1. Run `mypy` strict on `lab/` and fix the findings.
2. Fail CI below a coverage floor, for example 90% of lines, on `queue`, `policy`, `broker` and `sandbox`.
3. Register a safety marker and have CI count marked tests instead of name matches.

Done when. CI runs `mypy` clean, enforces the floor, and fails if marked safety tests drop.

Where: `pyproject.toml`, `.github/workflows/check.yml`

□ 2.4 Regression, race and crash tests

Effort: 3 days · Depends on: 1.6 · From: HL30, v1 2.4

Why. The existing suite passes (90 on Linux, 24 macOS tests skipped there), yet all eleven reproductions succeed. The defects live between steps and across connections.

Steps

1. Turn each reproduction, R01 to R11, into a regression test that checks the real effect.
2. Add race tests with two independent database connections.
3. Add a Hypothesis state machine that checks invariants after every step.
4. Add a crash test that kills a real process at random points, restarts and re-checks.

Done when. Every R01 to R11 test fails on the old code and passes after phase 1.

Where: `tests/`

□ 2.5 Turn on GitHub's security features

Effort: 1 hour · Depends on: None · From: v1 2.5 · [Start here](#)

Why. The repository is public, so these are free and catch leaked secrets and risky code early.

Steps

1. Turn on secret scanning and push protection.
2. Enable CodeQL default setup and add the OpenSSF Scorecard workflow.
3. Protect main so CI must pass before a merge.

Done when. All three show as active in the repository settings.

Where: Repository settings, Code security

□ 2.6 Keep claims and code in agreement

Effort: 0.5 day, then ongoing · Depends on: None · From: HL31 · [Start here](#)

Why. Several documents claim more than the code does: owner fencing, broker tiers and “the only way a worker touches anything real”. PLAN.md compares Kev’s 0.852 test score with Jev’s 0.857 development score. On the same development set the scores are 0.822 and 0.857. SUBSYSTEMS.md also lists Kev-9B at 421M parameters, which is Laya’s size.

Steps

1. Correct these claims now.
2. Update ARCHITECTURE, SECURITY and module comments in the same pull request as each change.
3. Link every claimed guarantee to its test and the platform it was tested on.

Done when. Every guarantee in the docs names the test that proves it.

Where: docs/PLAN.md:48, docs/SUBSYSTEMS.md:163, lab/broker.py docstring

Phase 3 Persistence and recovery

Weeks 5 to 6 · about 5.5 focused days · 4 items · Priority P1

Disciplined records without replacing SQLite. A backup counts only once a restore has worked.

□ 3.1 Version the schema and its invariants

Effort: 1 day · Depends on: 1.6 · From: HL13, v1 1.9

Why. CREATE TABLE IF NOT EXISTS never migrates an existing database. Execution attempts, action intents and operation receipts also need tables of their own.

Steps

1. Record the version in PRAGMA user_version and apply ordered migrations in one transaction each.
2. Add checks such as non-negative attempts, valid budgets and payload size limits.
3. Test upgrades against old database fixtures, including an interrupted migration.

```
version = conn.execute("PRAGMA user_version").fetchone()[0]
for n, migrate in enumerate(MIGRATIONS[version:], start=version + 1):
    with tx(conn):
        migrate(conn)
        conn.execute(f"PRAGMA user_version = {n}")
```

Done when. An old fixture upgrades with its data intact, and an interrupted migration rolls back cleanly.

□ 3.1 Version the schema and its invariants

Where: lab/schema.sql, lab/queue.py (_apply_schema)

□ 3.2 An audit log that cannot be quietly rewritten

Effort: 1.5 days · Depends on: 3.1 · From: HL15, v1 1.7 (corrected)

Why. Nothing stops edits or deletes in events, deleting a task cascades its history, and the broker logs nothing. A hash chain kept in the same writable database can be recomputed by whoever can write it, so it needs a copy kept elsewhere.

Steps

1. Remove the cascading deletes, and add triggers that refuse UPDATE and DELETE on events.
2. Allow writes only through the service, with restricted database permissions.
3. Log every broker call: tool, parameter hash, lease generation, decision and result status.
4. Export signed checkpoints of the latest hash and count to a destination the lab account cannot write.

```
CREATE TRIGGER events_no_update BEFORE UPDATE ON events
BEGIN SELECT RAISE(ABORT, 'events are append-only'); END;
CREATE TRIGGER events_no_delete BEFORE DELETE ON events
BEGIN SELECT RAISE(ABORT, 'events are append-only'); END;
```

Done when. Editing an event is refused, and rewriting the whole chain is caught against the last outside checkpoint.

Where: lab/schema.sql:89, 109, 140

□ 3.3 Content-addressed artifacts and receipts

Effort: 2 days · Depends on: 1.5, 1.6 · From: HL15

Why. Workspaces are deleted after a task, and a file list without hashes cannot reproduce an experiment.

Steps

1. Stop the writer first, then ingest regular files through verified descriptors.
2. Store SHA-256, size, type, task, attempt, tool version, source lineage and retention class.
3. Refuse links, devices and sockets, and quarantine files that fail ingestion.

Done when. After a restore, artifact hashes verify, a task can be reconstructed, and no successful state points to a missing artifact.

Where: lab/broker.py (manifest, close_workspace)

□ 3.4 Continuous backup and restore drills

Effort: 1 day · Depends on: 1.9 · From: HL14, v1 5.3

Why. The external SSD is storage, not a backup. Backup quality is shown by a restore, not by files existing.

Steps

1. Replicate the database with Litestream, or snapshot it with SQLite's backup API, to a separate device the worker cannot delete.
2. Keep backups versioned and encrypted. Suggested targets to test: at most 24 hours of artifact loss and a two-hour restore.
3. Restore monthly into a fresh location, run an integrity check and verify artifact hashes.

```
dbs:
- path: /Users/lab/.local/share/home-lab/lab.db
  replica:
    url: sftp://backup@<host>:22/home-lab/lab.db
```

□ 3.4 Continuous backup and restore drills

Done when. The monthly restore drill passes and its date is logged.

Where: ops/mac-mini-setup.md (Storage)

Phase 4 Before reading the outside world

Weeks 7 to 10 · about 17.5 focused days · 8 items · Priority P1, gate for issue #39

These controls must hold before the observation plane reads anything from outside or a real account is connected. Keep shell network access and broad credentials disabled until they pass.

□ 4.1 Write the authority rules into an ADR

Effort: 0.5 day · Depends on: None · From: v1 3.1, review page 5

Why. Meta's Agents Rule of Two allows a session at most two of three properties: untrusted input, sensitive data, and the power to act or communicate. Effective authority is the intersection of operator delegation, agent scope, task scope, input origin, tool policy and current approval.

Steps

1. Write ADR 0006 with a table of each plane and the two properties it may hold.
2. State that a model or classifier may suggest a route but never widen that intersection.
3. Encode the table in policy so no task can hold all three.

Done when. A test that gives one task untrusted input, a secret and an external tool is refused.

Where: docs/decisions/0004-operating-system.md

□ 4.2 Carry origin and authority ceilings through every task

Effort: 3 days · Depends on: 1.1, 1.2, 3.1 · From: HL09, v1 3.2

Why. Tasks carry a parent id but no record of where their input came from. External text may supply evidence, but must never define a tool grant, a destination or a policy.

Steps

1. Store source type, source id and hash, acquisition time, parent task, delegation and sensitivity.
2. Carry lineage through summaries, child tasks and retrieved memory.
3. Let a person approve one reviewed effect without erasing its untrusted origin.
4. Route untrusted text through a component whose output is a fixed schema, and test it. A provenance field or warning banner alone is not protection.

Done when. In tests, a malicious document cannot change authority, and returned content keeps its provenance.

Where: lab/schema.sql (tasks), lab/policy.py

□ 4.3 A narrow fetch gateway

Effort: 3 days · Depends on: 1.2, 4.2 · From: HL10, v1 3.3

Why. A domain allowlist alone cannot stop uploads to an attacker's account on an allowed service, and redirects or DNS changes can reach private addresses.

Steps

1. Keep shell network access denied. Add a trusted fetch connector with allowlisted schemes, hosts and ports, plus response size and time limits.
2. Block loopback, private, link-local and cloud metadata addresses. Resolve and validate addresses, and re-check every redirect.
3. Restrict method, destination path and allowed data, not just the domain.
4. If a shell tool ever needs network, reuse the srt pattern: the sandbox may talk only to a local proxy that enforces the allowlist.

Done when. Redirects and DNS changes cannot reach protected addresses, other hosts fail, and every attempt is logged.

Where: github.com/roshanaryal1/home-lab/issues/14

□ 4.4 Secret-holding connectors, one destination each

Effort: 4 days · Depends on: 4.3 · From: HL11, v1 3.4 (corrected)

Why. The model should never see a token. A general proxy that adds credentials to any allowed request would let the model sign requests of its own choosing, so each connector serves one destination.

Steps

1. Run a trusted connector per destination, holding Keychain credentials in an account the lab user cannot read.
2. Prefer short-lived, scoped tokens, for example GitHub App tokens that expire after an hour.
3. Give the worker a credential alias and the result, never the token bytes.
4. Log each use by rule id, and scan dummy token canaries through the whole path.

Done when. A worker cannot obtain or redirect a dummy credential, and revoking a grant blocks the next request.

Where: github.com/roshanaryal1/home-lab/issues/15

□ 4.5 Take approval power away from the agent account

Effort: 2 days · Depends on: 1.2 · From: v1 3.5, HL16, HL29

Why. The supervisor and the approval command share one database, so anything running as the agent user could write an approval. The dedicated lab account does not exist yet, and --by is free text.

Steps

1. Create the non-admin lab account, and install the code where lab can read it but not change it.
2. Authenticate the operator through OS identity, and treat --by as an audit label only.
3. Accept approvals only when signed by, or written to a store owned by, the operator account.
4. In the CLI, reject ambiguous id prefixes, preview exact targets and diffs, escape terminal control characters, and redact through lists as well as objects.

Done when. A process running as lab cannot create a valid approval or change the code, and an ambiguous prefix is refused.

Where: lab/cli.py:95, 128, 151, ops/mac-mini-setup.md

□ 4.6 Isolation for untrusted code

Effort: 1 day · Depends on: None · From: v1 3.6, review page 4

Why. sandbox-exec is deprecated but works. Apple's container 1.0 runs each Linux container in its own small virtual machine, but it runs Linux guests, so it cannot replace every macOS command.

Steps

1. Run Linux-compatible untrusted code in a disposable container with only the workspace mounted.
2. Keep native macOS tasks on Seatbelt through a separate, narrowly allowed path.
3. Measure memory and start-up time against the 3.8 GB margin.

Done when. A short ADR records the measured cost and which tasks use which boundary.

Where: lab/sandbox.py (docstring), SECURITY.md

□ 4.7 Test with benign and hostile tasks together

Effort: 3 days · Depends on: 4.2, a served model · From: HL30, v1 3.7

Why. PLAN.md's phase 3 gate says injected text must not become instructions. Grade what actually happened, not the transcript. A reply saying "I refused" does not count if a tool still ran.

Steps

1. Run AgentDojo, which pairs useful tasks with injection attacks, against the local model.
2. Add lab-specific cases: a fetched page asks the agent to email a file, change a route or call a new tool.
3. Grade real files, database state and simulated connector effects.

Done when. Attack success is zero for any action that needs approval, with results stored alongside the versions used.

Trade-off. Sampling has limits. Zero failures in 50 trials still allows a failure rate up to about 5.8% at 95% confidence, and about 1.0% at 300. It says nothing about adaptive attacks.

Where: docs/PLAN.md (Phase 3 gate)

□ 4.8 Write the threat model

Effort: 1 day · Depends on: 4.1 · From: v1 3.8

Why. The OWASP Top 10 for Agentic Applications (December 2025) names ten agent risks. One page mapping each to a control and a test shows reviewers the design is complete.

Steps

1. Create THREATS.md with one row per risk.
2. For each: the control, the test that proves it, and the open issue where a gap remains.

Done when. Every row has a control and either a test or an issue number.

Where: SECURITY.md

Phase 5 Prove the model on the M6

Weeks 11 to 12, needs the Mac mini · about 7 focused days · 3 items · Priority P1

Choose the model by completed work inside the measured memory budget, not by headline speed. Start as soon as the Mac is on the desk.

□ 5.1 A bounded native model adapter

Effort: 2 days · Depends on: 1.1, 1.10 · From: HL18, v1 4.3

Why. Untracked model, tokenizer or parser changes quietly change results, and a silently repaired tool call can carry a dangerous argument.

Steps

1. Freeze model and tokenizer revisions, chat template, the Qwen3-Coder tool-call parser, quantization and generation settings.
2. Validate every tool call against a strict schema, and reject malformed calls instead of repairing them.
3. Run one heavy service with one admission controller that bounds prompt length, output tokens, time and resident models.

Done when. Malformed calls are refused in tests, and oversized requests are refused before they cause swapping.

Where: docs/decisions/0001-heavy-model.md

□ 5.2 Benchmark by completed work

Effort: 3 days · Depends on: 5.1, the M6 · From: HL19, v1 4.1

Why. The 16.7 GB, the 3.8 GB margin and the 6 to 12 times speed-up are planning figures, not measurements. Reading long agent prompts is compute-bound, so prompt speed matters.

Steps

1. Run identical realistic coding, research and tool-use tasks on Qwen and one smaller baseline.
2. Measure cold and warm load, time to first token, prefill, generation and total task time.
3. Measure host memory pressure, swap and peaks under realistic background load.
4. Start at 8K context and go to 32K and 64K only if limits allow. The estimate is 6.4 GB of cache at 64K.

```
mlx_lm.benchmark --model <repo> --prompt-tokens 8192 \  
--generation-tokens 512 --num-trials 5
```

Done when. ADR 0001 records the model with the best correct-task time inside the measured budget, and a safe default context.

Where: docs/decisions/0001-heavy-model.md

❑ 5.3 Constrained decoding for tool calls

Effort: 2 days · Depends on: 5.2 · From: v1 4.2

Why. Constrained decoding makes invalid tool calls impossible rather than unlikely. `mlx-vlm`'s server already enforces a JSON Schema while generating, through `llguidance`.

Steps

1. Add grammar-constrained decoding for tool calls and routing labels.
2. Re-run the tool-use tasks from 5.2 with and without it.
3. Keep it if correctness rises without a meaningful speed cost.

Done when. Constrained results are recorded next to the baseline.

Where: `docs/decisions/0001-heavy-model.md`

Phase 6 Run it unattended

Week 13 · about 5.5 focused days · 4 items · Priority P1

Everything needed to recover from power cuts, crashes and updates without you in the room.

❑ 6.1 Decide power and disk-encryption behavior

Effort: 0.5 day plus a UPS · Depends on: None · From: v1 5.1

Why. With FileVault on, the Mac waits at the password screen after every reboot. FileVault protects powered-off storage, not files a running worker can read.

Steps

1. Turn on automatic start after a power failure, and add a small UPS.
2. Choose FileVault on with outside alerts and a way into the home network, or off with the Mac physically secure.
3. Write the choice into `ops/mac-mini-setup.md`.

```
sudo pmset -a autorestart 1
```

Done when. A test pull of the plug ends with the lab running again, or an alert on your phone.

Where: `ops/mac-mini-setup.md`

❑ 6.2 launchd service with a singleton lock and watchdog

Effort: 2 days · Depends on: 1.3, 4.5 · From: v1 5.2, HL16

Why. KeepAlive restarts a process that dies, not one that hangs, and the setup checklist has not been run.

Steps

1. Install a pinned LaunchDaemon that runs as lab, with bounded restart backoff and a singleton lock.
2. Add a separate watchdog that restarts the supervisor when its heartbeat goes stale.
3. Pause safely when a required drive or the sandbox is missing.
4. Test boot, logout, sleep and wake, drive removal and loss of the model process.

Done when. Killing or freezing the supervisor ends in a restart within two minutes, and a second supervisor refuses to start.

Where: `ops/mac-mini-setup.md` (Always-on)

❑ 6.3 Operator controls, alerts and a real emergency stop

Effort: 2 days · Depends on: 1.8, 3.3 · From: HL29, v1 5.4

Why. A watchdog on the same Mac cannot report its own power cut, and today no command pauses everything.

Steps

1. Show queue age, worker health, memory pressure, lease generation, current tool and approvals in the CLI.
2. Add pause, drain, cancel, stop and reconcile. Stop must revoke broker authority, not only set a flag.
3. Ping an external dead-man's switch every five minutes, and send phone alerts for approvals, stopped workers, low disk and memory pressure.

Done when. Stopping an active dummy task leaves no later tool effect, and unplugging the network raises an alert within ten minutes.

Where: README.md (Status, step 10)

❑ 6.4 Control macOS updates and self-test nightly

Effort: 1 day · Depends on: 6.2 · From: v1 5.5

Why. A new macOS version has already needed a sandbox fix once (#25), and the sandbox fails closed.

Steps

1. Turn off automatic major upgrades, and keep security responses on.
2. After any update, run the sandbox tests on the mini before resuming.
3. Run the suite nightly on the mini and send the result to your phone. Do not use a GitHub self-hosted runner, because the repository is public.

Done when. A nightly result arrives every morning.

Where: tests/test_sandbox.py

Phase 7 Ship one useful slice: issue #39

Week 14 · about 6.5 focused days · 3 items · Priority P1

One complete observe-to-draft workflow with explicit stopping points. No send or post connector is reachable in this slice.

❑ 7.1 One event-to-draft workflow

Effort: 3 days · Depends on: Phase 1, 3.3, 5.1 · From: HL24

Why. Starting with email, social feeds and browsing together multiplies risk. One known event stream proves the whole path first.

Steps

1. Use one event source, such as an approved merge in the lab's own repository.
2. Record the event id and revision, deduplicate, and create an ordinary queue proposal.
3. Apply a rate limit and a daily proposal budget.
4. Store sources, draft and hashes, then stop for review.

Done when. Replaying the same event produces one proposal, and every draft stops for a person.

Where: github.com/roshanarya11/home-lab/issues/39

□ 7.2 An evidence rubric that can say no

Effort: 1.5 days · Depends on: 4.2, 7.1 · From: HL25

Why. A router that must always pick post, blog or paper will promote thin evidence.

Steps

1. Define post, blog and paper candidate, plus insufficient evidence and no artifact.
2. Start with deterministic rules, and treat a paper route as a research candidate, not a quality claim.
3. Keep the rubric out of reach of injected source text.

Done when. Thin evidence routes to insufficient evidence, and injected text cannot change the rubric.

Where: docs/decisions/0004-operating-system.md

□ 7.3 Utility evaluations with full provenance

Effort: 2 days · Depends on: 5.2 · From: HL30, v1 6.2

Why. A result nobody can re-run is a claim, not evidence. Inspect runs evaluations with structured logs against any OpenAI-compatible server, including the local MLX server.

Steps

1. Start with 20 to 50 representative tasks, and repeat selected cases.
2. Save each run with commit, model revision, library and SQLite versions, macOS build and power settings.
3. Grade outcomes, and report failures and human intervention time.

Done when. The fixed task set runs repeatably, and any run can be re-run from its record.

Where: docs/PLAN.md (Phase 5)

Phase 8 Research and extensions (optional)

Weeks 15 to 18, then ongoing · about 18.5 focused days · 8 items · Priority P2, build only what evidence justifies

None of these is required for the first release. Add each one when an evaluation shows it solves a real limitation, and never with wider default authority.

□ 8.1 Pre-register the P3 measurement plan

Effort: 1 day · Depends on: Gate 4 · From: v1 6.1

Why. Writing the plan down before P3's six-week window protects the result from later doubt.

Steps

1. Write the hypotheses, metrics, failure categories and analysis plan.
2. Register it on OSF before week one, and link it from PIPELINE.md.

Done when. The registration is time-stamped before the first eval run.

Where: docs/PIPELINE.md

□ 8.2 Make each paper citable

Effort: 0.5 day · Depends on: 7.3 · From: v1 6.3

Why. A DOI lets reviewers cite and check the exact code and data behind a paper.

Steps

1. Add CITATION.cff, connect the repository to Zenodo, and release per paper.

Done when. The first paper release has a DOI.

Where: docs/PIPELINE.md

□ 8.3 Typed decision model as a shadow experiment

Effort: 3 days · Depends on: 5.2, 7.2 · From: HL20

Why. Typed output helps, but accuracy, calibration and authority are separate questions. Kev-9B needs about 19 GB in bf16, which cannot sit beside Qwen's 16.7 GB on 32 GB.

Steps

1. Build a baseline from deterministic rules and human labels.
2. Run candidates in shadow mode, making recommendations only.
3. Measure per-class errors, false promotion to paper, abstention, calibration, truncation, latency and peak memory on held-out items.

Done when. A typed "allow" or a confidence score never acts as permission, and adoption rests on a measured gain.

Where: docs/PLAN.md:35 to 64, docs/SUBSYSTEMS.md:163

□ 8.4 Inspectable memory with an FTS baseline

Effort: 3 days · Depends on: 4.2, 3.1 · From: HL21, HL22, v1 6.4

Why. Memory you cannot inspect or revoke becomes a path for poisoned content.

Steps

1. Store source id, hash, times, trust level, expiry and embedding version, with commands to inspect, correct, revoke and delete.
2. Start with FTS5, then compare vectors on a labelled set of real lab questions.
3. Pin sqlite-vec 0.1.9, avoid the alpha indexes, and load it only from a controller-owned path.

Done when. A revoked memory disappears from retrieval and from downstream drafts.

Where: docs/decisions/0003-memory.md

□ 8.5 A claim ledger with contradiction review

Effort: 3 days · Depends on: 3.3, 8.4 · From: HL23

Why. A source hash proves which bytes were used, not that they are true, and mirrored copies are not independent confirmation.

Steps

1. Keep source snapshot, locator, snippet, retrieval date and support or contradiction links per claim.
2. Require a contradiction pass and a missing-evidence list before a draft is reviewable.

Done when. An unsupported claim is flagged, and a reviewer can open the exact source behind any statement.

Where: docs/SUBSYSTEMS.md (research subsystem)

□ 8.6 Reviewed publishing with durable receipts

Effort: 3 days · Depends on: 1.4, 1.7, 4.4 · From: HL26

Why. Publishing is the riskiest action in the lab and the hardest to undo.

Steps

1. Show the exact destination and final draft hash for approval.
2. Reserve an operation id, send through the scoped connector, and store the provider receipt.
3. Reconcile when a response is lost, and make any edit require a new approval.

Done when. A lost response never leads to a duplicate post.

Where: docs/decisions/0004-operating-system.md

□ 8.7 Skills and executors with a rollback path

Effort: 3 days · Depends on: 4.7, 7.3 · From: HL27, HL28

Why. Learned skills and outside agent tools can quietly widen authority.

Steps

1. Treat skills as versioned artifacts with lineage, promoted only by trusted code or the operator.
2. Evaluate one coding or browser executor at a time, and reject any that needs an unsupervised shell or broad credentials.
3. Keep a known-good version and a one-step rollback.

Done when. A skill cannot lower its own permission tier, and rollback is tested.

Where: docs/SUBSYSTEMS.md (skill evolution)

□ 8.8 Tune only after correctness is stable

Effort: 2 days · Depends on: 5.2, 7.3 · From: HL32

Why. Optimizing before the guarantees hold hides regressions.

Steps

1. Measure queue wait, useful tasks per hour, tail latency, peak memory, thermal throttling and energy.
2. Profile before changing the database or model, and control background load when comparing runs.

Done when. Each tuning change shows a measured gain on the fixed task set.

Where: bench/, evals/

Decisions to make

Each has a real cost on both sides. The suggestion is where the evidence points today. The choice is yours, and each belongs in an ADR once made.

DECISION	OPTION A	OPTION B	SUGGESTED
Commit durability	FULL plus fullfsync: no lost commits, a few milliseconds slower	NORMAL: faster, can lose recent commits in a power cut	A. The retry and crash rules depend on it.
Worker isolation	Handlers inside the supervisor: simple, no boundary	Separate worker process and account: more plumbing, real boundary	B, from item 1.2.
Shell network access	Keep denied, fetch through the gateway	Proxy with a domain allowlist for shell tools	A now. Add B only for a real task.
Credentials	General proxy that adds credentials	One connector per destination	B. A general proxy lets the model choose what gets signed.
Disk encryption	FileVault on: password needed after reboots	FileVault off: restarts on its own	A with a UPS and outside alerts, unless unattended restart matters more.
Untrusted code	Seatbelt: no memory cost, deprecated tool	Apple container: VM boundary, Linux only	Container for Linux-compatible untrusted code.
Typed decision model	Adopt as the router now	Shadow experiment first	B. It recommends, it never authorizes.
Autonomy	Strict Rule of Two: more approvals	Looser: fewer interruptions, higher risk	Strict first. Relax only with evidence from 4.7.

Evidence

The deep review reported eleven reproductions. All eleven were re-run independently on 28 September 2026 against commit 08c01c7, with disposable data on Linux, and each behaved as reported. R08 needed an artificial pause, so its real window is narrow. R09 and R10 show unsafe host-side file handling, not an escape from the macOS kernel sandbox. Live Mac behavior still needs the acceptance tests on the Mac.

ID	WHAT HAPPENED	FIXED BY
R01	fs.delete ran with no approval, and the policy object was never called	1.1
R02	A different worker label read another task's workspace	1.2
R03	An owner that never leased a task marked it succeeded	1.3
R04	A stale process with the same owner name completed work and released the new lease	1.3
R05	A failed audit insert left the task succeeded, with no event and a live lease	1.6
R06	A task marked unsafe to repeat was requeued after an ordinary failure	1.7
R07	A policy error killed a worker, run() returned normally, and the task stayed leased	1.8
R08	An approval that expired between lookup and use was still spent	1.4
R09	A directory swapped for a symlink after the check sent a write outside the workspace	1.5
R10	A planted .sandbox.sb symlink made the supervisor overwrite an outside file	1.5
R11	Sandboxed commands receive the supervisor's full environment	1.10

Outside facts checked at source

- SQLite 3.7.0 to 3.51.2 carry the rare WAL-reset corruption bug. It is fixed in 3.51.3 (13 March 2026), with backports in 3.44.6 and 3.50.7. At least one project found uv-managed Python builds shipping SQLite 3.50.4.
- Kev-9B's model card gives 0.822 on the development set and 0.852 on the locked test, against Jev's 0.857 on the development set. The matched gap is 3.5 points, not half a point.
- Not verified: Laya's context sizes, and Kev-9B's memory of about 19 GB in bf16. The second is plausible, since 9 billion parameters at 2 bytes each is about 18 GB.

Checklist

Print these pages, or tick the boxes as pull requests merge. Phase 8 items are optional.

ID	Item	Effort	Depends on
☐ 1.1	Enforce policy on every tool call	1.5 days	None
☐ 1.2	Bind identity to a trusted execution context	3 days	1.1
☐ 1.3	Replace owner-name fencing with lease tokens	2 days	None
☐ 1.4	Approve an immutable action intent	2.5 days	1.1, 1.3
☐ 1.5	Make file handling race-safe and protect the profile	2.5 days	1.2 design
☐ 1.6	Commit each state change with its audit record	2 days	1.3
☐ 1.7	Retry only when the outcome is known	2.5 days	1.4, 1.6
☐ 1.8	Stop the work, not only its result	2 days	1.2, 1.3
☐ 1.9	Durable commits on a safe SQLite	0.5 day	None
☐ 1.10	Clean environment and strict limits for every request	1.5 days	1.1, 1.8
☐ 2.1	Commit the lockfile and pinned Python	1 hour	None
☐ 2.2	Harden the CI pipeline	2 hours	2.1
☐ 2.3	Type checks, coverage floors and a safety marker	1.5 days	2.1
☐ 2.4	Regression, race and crash tests	3 days	1.6
☐ 2.5	Turn on GitHub's security features	1 hour	None
☐ 2.6	Keep claims and code in agreement	0.5 day, then ongoing	None
☐ 3.1	Version the schema and its invariants	1 day	1.6
☐ 3.2	An audit log that cannot be quietly rewritten	1.5 days	3.1
☐ 3.3	Content-addressed artifacts and receipts	2 days	1.5, 1.6
☐ 3.4	Continuous backup and restore drills	1 day	1.9
☐ 4.1	Write the authority rules into an ADR	0.5 day	None
☐ 4.2	Carry origin and authority ceilings through every task	3 days	1.1, 1.2, 3.1
☐ 4.3	A narrow fetch gateway	3 days	1.2, 4.2
☐ 4.4	Secret-holding connectors, one destination each	4 days	4.3
☐ 4.5	Take approval power away from the agent account	2 days	1.2
☐ 4.6	Isolation for untrusted code	1 day	None
☐ 4.7	Test with benign and hostile tasks together	3 days	4.2, a served model
☐ 4.8	Write the threat model	1 day	4.1
☐ 5.1	A bounded native model adapter	2 days	1.1, 1.10
☐ 5.2	Benchmark by completed work	3 days	5.1, the M6
☐ 5.3	Constrained decoding for tool calls	2 days	5.2
☐ 6.1	Decide power and disk-encryption behavior	0.5 day plus a UPS	None
☐ 6.2	launchd service with a singleton lock and watchdog	2 days	1.3, 4.5
☐ 6.3	Operator controls, alerts and a real emergency stop	2 days	1.8, 3.3
☐ 6.4	Control macOS updates and self-test nightly	1 day	6.2
☐ 7.1	One event-to-draft workflow	3 days	Phase 1, 3.3, 5.1
☐ 7.2	An evidence rubric that can say no	1.5 days	4.2, 7.1
☐ 7.3	Utility evaluations with full provenance	2 days	5.2
☐ 8.1	Pre-register the P3 measurement plan	1 day	Gate 4
☐ 8.2	Make each paper citable	0.5 day	7.3
☐ 8.3	Typed decision model as a shadow experiment	3 days	5.2, 7.2
☐ 8.4	Inspectable memory with an FTS baseline	3 days	4.2, 3.1

ID	Item	Effort	Depends on
☐ 8.5	A claim ledger with contradiction review	3 days	3.3, 8.4
☐ 8.6	Reviewed publishing with durable receipts	3 days	1.4, 1.7, 4.4
☐ 8.7	Skills and executors with a rollback path	3 days	4.7, 7.3
☐ 8.8	Tune only after correctness is stable	2 days	5.2, 7.3

Sources

Research behind this plan, checked on 28 September 2026.

- Home Lab Deep Review, repository review and primary-source research, 28 September 2026 (uploaded PDF).
- Meta AI, Agents Rule of Two (October 2025). [Link](#)
- Design Patterns for Securing LLM Agents against Prompt Injections (2025). [Link](#)
- CaMeL: capability-based defense against prompt injection (2025). [Link](#)
- AgentDojo benchmark. [Link](#)
- OWASP Top 10 for Agentic Applications, summary. [Link](#)
- OWASP SSRF Prevention Cheat Sheet. [Link](#)
- OWASP Secrets Management Cheat Sheet. [Link](#)
- MITRE CWE-367, check and use race. [Link](#)
- AWS Builders Library, making retries safe with idempotent APIs. [Link](#)
- RFC 8785, JSON Canonicalization Scheme. [Link](#)
- SQLite Write-Ahead Logging and the WAL-reset bug. [Link](#)
- SQLite Online Backup API. [Link](#)
- SQLite on macOS and fullfsync. [Link](#)
- uv-managed Python shipping an affected SQLite. [Link](#)
- Python subprocess documentation. [Link](#)
- Anthropic Sandbox Runtime (srt). [Link](#)
- Apple container. [Link](#)
- Litestream. [Link](#)
- mlx-lm. [Link](#)
- mlx-vlm structured output with llguidance. [Link](#)
- Qwen3-Coder-30B-A3B-Instruct model card. [Link](#)
- Kev-9B model card. [Link](#)
- Inspect evaluation framework. [Link](#)
- Pinning GitHub Actions by commit SHA. [Link](#)
- GitHub Actions runner images. [Link](#)
- FileVault on unattended Mac servers. [Link](#)
- sqlite-vec releases. [Link](#)